Module 2: Training-Loop Anatomy

Every State Change Behind One Optimizer Update

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. One optimizer update
2. Ordering invariants
3. Evaluation and recovery

Guiding question

Which state changes at each line of a training loop, and what silently breaks when the order changes?

What should be clear by the end?

1. Separate graph construction, gradient accumulation, and parameter updates.
2. Compute an effective token batch under accumulation and data parallelism.
3. Place AMP, clipping, scheduling, evaluation, and checkpointing correctly.
4. Audit a loop through invariants instead of trusting a decreasing loss.

PART 1

One optimizer update

1. One optimizer update
2. Ordering invariants
3. Evaluation and recovery

Five lines hide four kinds of state

1. clear parameter gradient buffers
2. build a forward computation graph
3. accumulate derivatives into `.grad`
4. update parameters and optimizer moments

Minimal loop

```
zero_grad()
logits, loss =
model(x,y)
loss.backward()
optimizer.step()
```

Crucial distinction

`backward()` computes gradients. Only `optimizer.step()` changes weights.

Source: The Annotated PyTorch Training Loop

Backward accumulates into persistent buffers

For each parameter θ_i, autograd adds the current derivative:

$$\theta_i.\text{grad} \;\; += \;\; \frac{\partial \mathcal{L}}{\partial \theta_i}$$

This behavior enables gradient accumulation, but it also makes a missing reset dangerous.

Why `set_to_none=True`?

Deallocate buffers rather than write zeros across every element. The next backward pass recreates them.

Diagnostic

Log whether gradients are `None` before the first backward pass.

Gradient accumulation changes update frequency

$$N_{\text{tokens/update}} = B_{\text{micro}}\, T\, K\, W$$

- K micro-batches contribute to one update.
- Divide each equal-size micro-loss by K.
- Clip and step only at the update boundary.

Worked count

$B_{\text{micro}} = 4, T = 1024, K = 8, W = 2$

$N = 65{,}536$ tokens per optimizer update

Unequal micro-batches

Normalize by the number of valid tokens, not blindly by the number of micro-steps.

AdamW separates optimization from decay

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

Usually decayed

Linear and embedding matrices with two or more dimensions.

Usually not decayed

Biases and normalization scales, commonly one-dimensional tensors.

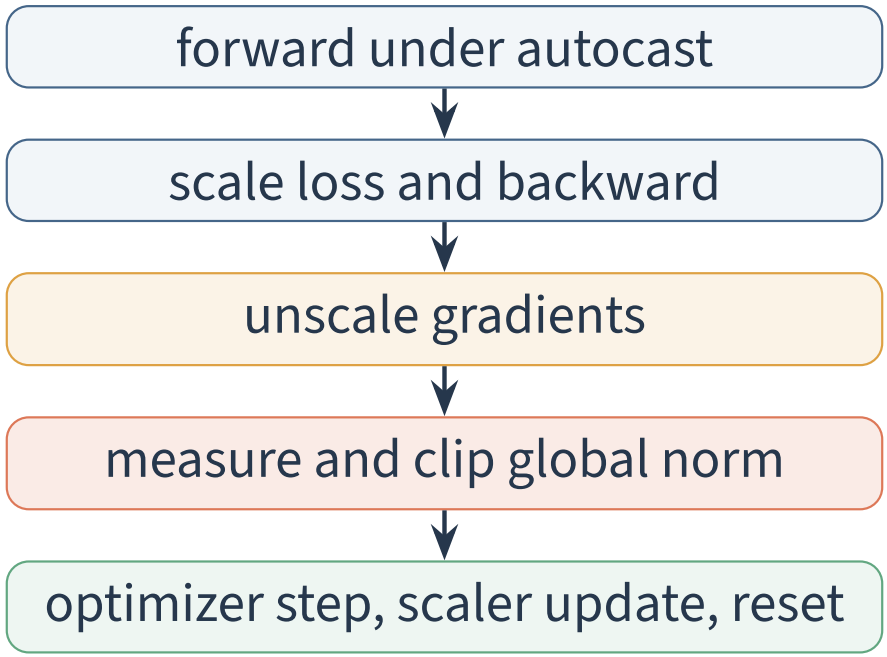
Audit the partition: every trainable parameter must appear exactly once.

PART 2

Ordering invariants

1. One optimizer update
2. Ordering invariants
3. Evaluation and recovery

Mixed precision adds one non-negotiable ordering rule



Why unscale first?

Clipping scaled gradients measures a fictitious norm and changes the effective threshold.

Global clipping preserves direction

$$\|g\|_2 = \sqrt{\sum_i \|g_i\|_2^2}$$

$$g_i \leftarrow g_i \cdot \min\left(1, \frac{M}{\|g\|_2 + \epsilon}\right)$$

- All parameter gradients receive the same multiplier.
- Log the pre-clip norm and whether clipping activated.
- Frequent saturation is evidence of instability, not proof that clipping fixed it.

The scheduler counts optimizer updates

$$\eta_t = \begin{cases} \eta_{\max} \frac{t+1}{T_w+1} & t < T_w \\ \eta_{\min} + \frac{1+\cos(\pi r)}{2}(\eta_{\max} - \eta_{\min}) & t \geq T_w \end{cases}$$

where $r = (t - T_w)/(T_{\max} - T_w)$.

Warmup

Limits early updates while activations and optimizer moments stabilize.

Silent failure

Stepping once per micro-batch compresses the schedule by factor K.

A catalog of loops that run but lie

Mistake	Consequence
optimizer built before device move	optimizer may reference discarded parameters
reset after backward	useful gradients are erased
clipping before backward	clipping is a no-op
clipping after step	the dangerous update already happened
validation without no-grad	graphs consume memory needlessly
log tensors instead of scalars	retained graphs create a memory leak
omit `train()` after evaluation	dropout stays disabled

Checkpoint: repair the step

Think, then compare

Put these operations in order: clip, autocast forward, scaler update, scaled backward, unscale, optimizer step, scheduler step, gradient reset. Which operations occur only once per accumulated update?

PART 3

Evaluation and recovery

1. One optimizer update
2. Ordering invariants
3. Evaluation and recovery

Evaluation changes mode, not parameters

Required state

`model.eval()` changes dropout behavior.

`torch.no_grad()` disables graph construction.

Stable estimate

Evaluate a fixed validation token budget. Aggregate loss by valid-token count.

Return path

Call `model.train()` before the next training batch.

An exact checkpoint captures the whole experiment state

Numerical state

model parameters
optimizer moments
AMP scaler
scheduler or global step

Data and randomness state

sampler position
Python, NumPy, CPU and CUDA RNGs
configuration and code version

Recovery test

An uninterrupted run and a save-resume run should produce the same next batch, learning rate, loss, and update within the declared tolerance.

A loop audit asks for evidence at boundaries

1. Count valid tokens per update.
2. Verify each parameter belongs to one optimizer group.
3. Log learning rate and pre-clip gradient norm at update boundaries.
4. Separate training loss, validation loss, throughput, and memory.
5. Run the exact-resume comparison before long training.

The durable ideas

1. A training loop is a sequence of state transitions, not a four-line ritual.
2. Accumulation changes the batch while clipping and scheduling remain update-level operations.
3. AMP is safe only when gradients are unscaled before norm measurement and clipping.
4. Validation and checkpoint recovery need explicit invariants of their own.